3D Autonomous Underwater Navigation: Optimizing Vector Field Histograms via Evolution Strategies in Gazebo

Adriano Semerano

Sapienza University of Rome
semerano.2160754@studenti.uniroma1.it

Abstract

This report details the development and optimization of an autonomous navigation system for the Tethys Autonomous Underwater Vehicle (AUV) within a Gazebo-based simulated environment. Rather than relying on traditional manual tuning of geometric controllers, this study applies OpenAI Evolution Strategies (ES) to automate the parameterization of a 3D Vector Field Histogram (VFH) algorithm. By removing hardware-level pitch constraints and applying a 3D Euclidean reward function, the VFH controller is adapted to support unconstrained 3D movement. The report outlines the generation of the simulation environments, the architectural adaptations of the VFH node in ROS 2, and the performance analysis of the evolutionary training over successive generations.

1 Introduction

Autonomous Underwater Vehicles (AUVs) operate in complex, unstructured 3D environments where reliable, real-time navigation is a primary requirement. Traditionally, obstacle avoidance and local path planning have relied on geometric controllers, such as the Vector Field Histogram (VFH) [1]. While the VFH method has been widely used for 2D planar navigation by providing continuous steering responses, its extension to 3D underwater scenarios presents implementation challenges. These controllers often require manual tuning and rely on hardcoded safety constraints—such as pitch limitations and high dampening factors—which can restrict the vehicle's maneuverability in spatial tasks.

To address the tuning of these parameters, this project applies an automated optimization framework utilizing Evolution Strategies (ES) [4]. As a scalable alternative to standard Markov Decision Process-based Reinforcement Learning, OpenAI ES operates as a gradient-free optimizer that perturbs policy parameters directly. This characteristic allows it to handle the discontinuous nature of the VFH cost function and manage long time horizons without relying on value function approximations.

In this study, a Tethys AUV is simulated within a Gazebo-based 3D environment. By removing the restrictive hardcoded constraints within the VFH C++ node and integrating it with a Python-based ES orchestrator, the algorithm maps sensory inputs (from a 3D Sonar PointCloud and altimeter) to 3D movement commands. By evaluating this evolutionary process in high-density obstacle fields, the project investigates how evolutionary optimization algorithms can be utilized to automate the parameter tuning of traditional geometric controllers, offering an alternative to manual calibration.

The complete source code, procedural generation scripts, and simulation demo videos for this project are open-source and publicly available at: `https://github.com/AdriSemerano/AFC_Tethys_Drone_Semerano.git`.

2 Related Work

Local path planning and obstacle avoidance for autonomous agents have been extensively studied. Early approaches to real-time navigation often relied on Artificial Potential Fields (APF), which treat the robot as a particle under the influence of an attractive force towards the goal and repulsive forces from obstacles. To mitigate inherent issues such as local minima and unstable oscillations in narrow corridors, Borenstein and Koren proposed the VFH algorithm [1]. By maintaining a statistical representation of the environment in a spatial grid and converting it into a 1D polar histogram, the VFH algorithm allows for continuous, fast, and non-oscillatory steering.

With the proliferation of Unmanned Aerial Vehicles (UAVs) and Autonomous Underwater Vehicles (AUVs), researchers have extended classical planar algorithms into spatial dimensions. Variations such as the 3D-VFH [2] convert the obstacle representation into a spherical or 2D polar histogram (pitch and yaw). While effective, these spatial extensions significantly increase the complexity of the controller. Geometric algorithms rely heavily on heuristic tuning of interconnected weights—such as target direction attraction, previous direction inertia, and obstacle repulsion magnitudes [3]. For complex underwater dynamics, manual parameterization often leads to suboptimal behavior and requires rigid hardware limits to maintain stability.

In recent years, learning-based methods have been explored to automate the derivation of navigation policies. Deep Reinforcement Learning (DRL) algorithms frequently replace geometric controllers entirely, mapping

sensor inputs directly to motor commands. However, DRL approaches are often sensitive to hyperparameters and reward scaling. As an alternative, Evolution Strategies (ES) [4] provide a scalable, gradient-free approach to optimization. By perturbing parameters across a population and utilizing the collective return, ES evaluates the fitness of a system without requiring differentiable pathways or value function approximations.

This project intersects these domains by utilizing ES not to replace the VFH algorithm, but to automate its parameterization. By treating the weights and thresholds of a 3D-VFH controller as an evolutionary genome, this study investigates how automated optimization can adapt a geometric algorithm to unconstrained spatial environments.

3 Methodology

3.1 Simulation Environment

The training and evaluation of the VFH policy are conducted within a simulated Gazebo environment, integrated with ROS 2 Humble. The custom ROS 2 package, `uuv_random_policy`, is built using CMake and manages the execution of the VFH node, depending on core libraries such as `rclcpp`, `sensor_msgs`, `nav_msgs`, and `ros_gz_interfaces`.

Tethys AUV Model The autonomous agent is based on the Tethys AUV model, equipped with specialized Gazebo plugins to simulate underwater physics, including buoyancy, hydrodynamics, and lift-drag effects on the vertical and horizontal fins. To perceive the environment, a custom sensor payload is attached to the vehicle:

- A 3D GPU Lidar (functioning as a simulated 3D sonar) that publishes 72 horizontal samples and 30 vertical samples, with a detection range between 0.3 and 40.0 meters.
- A downward-facing altimeter (1D Lidar) to measure the distance to the seabed, up to a maximum range of 100.0 meters.
- An Inertial Measurement Unit (IMU) operating at an update rate of 50 Hz.

ROS-Gazebo Communication Bridge To enable communication between the Gazebo simulator and the ROS 2 ecosystem, the `ros_gz_bridge` package is utilized. This component acts as a bidirectional interface, translating Gazebo transport messages into standard ROS 2 message types. Specifically, it routes sensor data—such as the 3D PointCloud from the GPU Lidar, altimeter laser scans, and odometry—from the simulated environment to the VFH control node. Concurrently, it transmits the computed actuator commands, including vertical and horizontal fin positions as well as propeller thrust, back to the Gazebo physics engine to close the control loop.

Procedural World Generation To expose the algorithm to diverse spatial challenges, the simulation utilizes a custom procedural generation script. The base world features a 500x500 meter deep seabed with large perimeter banks, dunes, and a central spire. The script populates an arena of 200.0 meters with various geometric obstacles:

- 100 spheres with radii ranging from 1.0 to 5.0 meters.
- 40 cylindrical pillars with lengths up to 120.0 meters.
- 120 boxes and 100 cylindrical beams.

The generation system supports different difficulty modes. In the default "random" mode, the target waypoint is placed at a random altitude, whereas in "canyon" mode, the objective is positioned within a central crevasse at negative altitudes (between -30.0 and -15.0 meters). A safety limit of 150.0 meters ensures the AUV spawns within a valid operational area, and a 20.0-meter safe zone radius prevents obstacles from overlapping with the spawn point and waypoints. Finally, the script automatically generates an executable bash script that spawns the AUV in the simulator using the correct spatial coordinates and yaw orientation.

3.2 3D Vector Field Histogram Node

The local obstacle avoidance policy is implemented as a custom C++ ROS 2 node (`vfh_node_rl`). The algorithm extends the classical 2D planar VFH to a three-dimensional spherical space by processing point cloud range data and generating continuous actuation commands for the AUV's propeller thrust and steering fins.

Spherical Histogram and Coordinate Transformation The environment's local spatial density is represented using a two-dimensional spherical histogram matrix $H \in \mathbb{R}^{N_{\text{yaw}} \times N_{\text{pitch}}}$, discretized into $N_{\text{yaw}} = 72$ azimuth sectors and $N_{\text{pitch}} = 36$ elevation sectors, corresponding to a spatial resolution of $5^\circ \times 5^\circ$.

For each point (x, y, z) received from the 3D sonar point cloud, range $d = \sqrt{x^2 + y^2 + z^2}$ is evaluated. Points falling outside the active detection range ($d < 0.5\,\text{m}$ or $d > d_{\text{safe}}$) are discarded. To ensure invariance to the AUV's instantaneous orientation, incoming points are transformed into a stabilized vehicle frame by unrolling the vehicle's roll (ϕ) and pitch (θ) angles:

$$y_{\text{unroll}} = y \cos\phi - z \sin\phi \tag{1}$$

$$z_{\text{unroll}} = y \sin\phi + z \cos\phi \tag{2}$$

$$x_{\text{stab}} = x \cos\theta + z_{\text{unroll}} \sin\theta \tag{3}$$

$$z_{\text{stab}} = -x \sin\theta + z_{\text{unroll}} \cos\theta \tag{4}$$

The polar directions $(\psi_{\text{pt}}, \theta_{\text{pt}})$ are then computed, mapping the stabilized coordinates to discrete grid indices $(y_{\text{idx}}, p_{\text{idx}})$. Each point contributes an obstacle magnitude m to the corresponding histogram bin, scaled inversely with distance:

$$m = vfh_a - \left(\frac{vfh_a}{d_{\text{safe}}}\right) d \tag{5}$$

where $vfh_a = 15.0$ is the base magnitude scaling factor. Additionally, if the downward altimeter detects seabed proximity ($z_{\text{alt}} < 15.0\,\text{m}$), an artificial obstacle density is dynamically injected into the lower pitch sectors to enforce ground avoidance.

Obstacle Expansion and Candidate Cost Evaluation To account for the physical dimensions of the AUV and safety margins, an obstacle expansion window w_{exp} applies a max-pooling operation across neighboring sectors:

$$H_{\text{exp}}(y, p) = \max_{|i| \le w_{\text{exp}}, |j| \le w_{\text{exp}}} H(y + i, p + j) \tag{6}$$

For every direction candidate $c = (\psi_c, \theta_c)$ in the expanded spherical grid, a scalar cost function $G(c)$ is evaluated:

$$\begin{aligned} G(c) = {} & W_{\text{TARGET_DIR}} \cdot |\Delta\psi_{\text{target}}| + W_{\text{PITCH_DIR}} \cdot |\Delta\theta_{\text{target}}| + \\ & W_{\text{OBS}} \cdot H_{\text{exp}}(y_c, p_c) + W_{\text{PREV_DIR}} \cdot |\Delta\psi_{\text{prev}}| + \\ & W_{\text{PITCH_STABILITY}} \cdot |\theta_c| \end{aligned} \tag{7}$$

where $\Delta\psi_{\text{target}}$ and $\Delta\theta_{\text{target}}$ represent the angular alignment errors relative to the local goal waypoint, $\Delta\psi_{\text{prev}}$ penalizes sudden changes from the previous chosen yaw direction, and $|\theta_c|$ penalizes steep pitch angles to maintain vertical attitude stability.

Stochastic Candidate Selection Unlike deterministic VFH implementations that rigidly select $\arg\min_c G(c)$, this node employs a stochastic Boltzmann (Softmax) selection scheme. The selection probability $P(c)$ for each candidate direction is defined as:

$$P(c) = \frac{\exp\left(-\beta_{\text{temp}} \cdot (G(c) - G_{\min})\right)}{\sum_k \exp\left(-\beta_{\text{temp}} \cdot (G(k) - G_{\min})\right)} \tag{8}$$

where $G_{\min} = \min_k G(k)$ and $\beta_{\text{temp}} = 2.0$ controls the distribution temperature. Sampling candidate trajectories stochastically introduces decision noise that aids in escaping local minima (such as U-shaped obstacles or narrow dead-ends) without requiring complex global path replanners.

Kinematics and Low-Level Fin Control Once a direction $(\psi_{\text{chosen}}, \theta_{\text{chosen}})$ is sampled, proportional controllers evaluate the raw control signals for vertical (v_{raw}) and horizontal (h_{raw}) fin angles based on tracking errors. To counteract hydrodynamic skidding and physical water drag effects, commands are passed through a low-pass smoothing filter before actuation:

$$u_v(t) = u_v(t-1) + \alpha_{\text{smooth}} \cdot (\text{clamp}(v_{\text{raw}}, \pm\theta_{\max}) - u_v(t-1)) \tag{9}$$

$$u_h(t) = u_h(t-1) + \alpha_{\text{smooth}} \cdot (\text{clamp}(h_{\text{raw}}, \pm\theta_{\max}) - u_h(t-1)) \tag{10}$$

where $\theta_{\max} = 0.2618\,\text{rad}$ ($\approx 15°$) represents the physical mechanical limit of the fins, and $\alpha_{\text{smooth}} = 0.35$ provides command dampening. Propeller thrust is scaled proportionally to the chosen trajectory cost:

$$T = \text{clamp}\left(T_{\min} + G_{\text{best}} \cdot W_{\text{THRUST}}, T_{\min}, T_{\max}\right) \tag{11}$$

bounded between $T_{\min} = -40.0\,\text{N}$ and $T_{\max} = 30.0\,\text{N}$. Crucially, hardware-level geometric pitch limits were removed from the tracking error calculation, unlocking full 90° spatial maneuverability.

Reactive Design Strategy vs. Persistent Volumetric Mapping The implementation intentionally operates as a purely reactive local controller utilizing a short-term active sensor window, omitting long-term volumetric mapping architectures (e.g., Octomap). This architectural choice is motivated by two key factors:

1. *Algorithmic Focus:* The primary objective is evaluating reflexive obstacle avoidance under evolutionary optimization, testing the AUV's capacity to navigate complex fields based on instantaneous 3D spatial perceptions.

2. *Computational Resource Efficiency:* Conducting long-term evolutionary training involving thousands of simulation rollouts requires strict resource management. Omitting persistent volumetric memory mapping eliminates cumulative memory overhead and avoids potential RAM leak bottlenecks within virtualized execution environments (e.g., Hyper-V instances with 16 GB RAM allocation), ensuring high execution stability over multi-day training sessions.

Evolvable Parameter Set In total, 8 core internal parameters governing perception, expansion, cost weights, and thrust scaling are exposed to the external evolutionary optimizer:

$$\boldsymbol{\theta}_{\text{vfh}} = [d_{\text{safe}}, w_{\text{exp}}, W_{\text{TARGET_DIR}}, W_{\text{PITCH_DIR}}, W_{\text{OBS}}, W_{\text{PREV_DIR}}, W_{\text{P}} \tag{12}$$

3.3 Evolution Strategies Framework

To automate the tuning of the 8 core parameters ($\boldsymbol{\theta}_{\text{vfh}}$) of the VFH node, this study implements an OpenAI Evolution Strategies (ES) framework integrated with the Adam

optimizer [4]. ES provides a highly scalable, gradient-free approach well-suited for the discontinuous nature of obstacle avoidance policies and the rigid non-differentiable physics of the Gazebo environment.

Population Generation and Perturbation For each training generation g, a population of $N = 8$ candidate solutions (clones) is generated by perturbing the current optimal parameter vector $\boldsymbol{\theta}_g$. The perturbations are sampled from a standard normal distribution, such that half of the population receives a positive perturbation and the other half receives a mirrored negative perturbation (antithetic sampling) to reduce gradient variance [4]:

$$\boldsymbol{\theta}_i = \text{clip}(\boldsymbol{\theta}_g + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}_i, \boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}) \tag{13}$$

where $\boldsymbol{\epsilon}_i \sim \mathcal{N}(0, I)$ for $i \in \{1, \ldots, N/2\}$, $\boldsymbol{\epsilon}_{i+N/2} = -\boldsymbol{\epsilon}_i$, and $\boldsymbol{\sigma}$ is a vector of mutation step sizes customized for the varied scales of the VFH parameters. The perturbed parameters are strictly clipped within valid operational bounds to prevent physical impossibilities (e.g., negative safety distances) [4].

Custom 3D Reward Function Each candidate parameter set $\boldsymbol{\theta}_i$ is evaluated within the simulated environment to calculate a fitness score, or cumulative reward R_i. Because the VFH algorithm acts as a local planner without a global path, the reward function is engineered to encourage progress toward the 3D target while heavily penalizing unsafe behaviors.

The core episode reward is structured around waypoint acquisition and collision penalties. If the AUV reaches a waypoint within the 4.0-meter tolerance, a large bonus is awarded (+15000), and the agent proceeds to the next waypoint. Conversely, any physical contact with the environment obstacles (measured by the Gazebo contact sensor) immediately terminates the episode and incurs a severe penalty (-20000). A time limit of 240.0 seconds is enforced per episode; exceeding this limit triggers a timeout penalty (-10000).

To provide a continuous dense gradient for the ES optimizer, a 3D Euclidean distance progress reward is calculated at each step. Let D_{start} be the distance from the AUV to the waypoint at the beginning of the segment, and D_{final} be the distance upon episode termination. The distance covered is rewarded proportionally:

$$R_{\text{dist}} = (D_{\text{start}} - D_{\text{final}}) \cdot 25.0 \quad \text{if } (D_{\text{start}} - D_{\text{final}}) > 0 \tag{14}$$

To promote efficient, direct routes and discourage cyclic behavior, minor penalties are applied based on elapsed episode time (-10.0 per elapsed second) and the total accumulated distance traveled by the AUV (-0.5 per meter).

Gradient Estimation and Adam Optimization Once all N clones are evaluated, their raw rewards $\boldsymbol{R} = [R_1, \ldots, R_N]$ are converted into rank-based utilities to ensure the optimization process remains robust against extreme reward outliers caused by environment variability. The rewards are sorted and mapped to normalized utilities $U_i \in [-0.5, 0.5]$ [4].

The estimated gradient $\nabla_{\boldsymbol{\theta}} J$ of the expected return is computed as a weighted sum of the initial perturbations:

$$\nabla_{\boldsymbol{\theta}} J \approx \frac{1}{N} \sum_{i=1}^{N} U_i \frac{\boldsymbol{\epsilon}_i}{\boldsymbol{\sigma}} \tag{15}$$

Rather than applying a simple Stochastic Gradient Descent update, the Adam optimizer is employed to compute the parameter step, utilizing exponential moving averages of the gradient (m) and the squared gradient (v). The parameters are updated as follows:

$$\boldsymbol{\theta}_{g+1} = \text{clip}\left(\boldsymbol{\theta}_g + \boldsymbol{\alpha} \odot \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}, \boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}\right) \tag{16}$$

where $\boldsymbol{\alpha}$ is the initial learning rate vector. To refine convergence as the policy matures, a decay factor of 0.85 is applied to both the learning rate $\boldsymbol{\alpha}$ and the mutation noise $\boldsymbol{\sigma}$ after the 17th generation.

4 Training Setup and Results

4.1 Training Procedure

The evolutionary training of the 3D-VFH node was conducted over 35 generations. To ensure the policy generalized robustly to different spatial geometries, a cross-evaluation approach was utilized. Within each generation, every clone $\boldsymbol{\theta}_i$ was evaluated across a sequence of 10 distinct, procedurally generated environments. These environments alternated evenly between the standard "random" mode (featuring unconstrained vertical targets) and the challenging "canyon" mode (forcing navigation through negative altitude crevasses). The final fitness R_i for a clone was computed as the mean reward across all 10 environmental rollouts.

The training framework required re-initializing the ROS-Gazebo bridge for each episode. Due to the high computational demand of resetting 3D point cloud sensors and simulating complex hydrodynamics, a Python-based orchestrator managed the lifecycle of the simulation, ensuring the Gazebo physics engine was cleanly initialized and terminated for every rollout.

Given the extensive time required for multi-environment evaluations, a micro-checkpointing system was implemented. The orchestrator serialized the state of the ES optimizer, including the current generation, perturbation matrices, and cumulative metrics, after every individual rollout. This mechanism guaranteed that the training process could automatically resume from the exact environment and clone index in the event of a system interruption or memory leak within the virtualized training environment.

4.2 Performance Analysis

The evolutionary training process optimized the 8 core VFH parameters over 35 generations. Table 1 summarizes the performance metrics of the best-performing clone (agent) at selected key intervals throughout the training process. The data illustrates a clear progression in the agent's ability to navigate the 3D environments.

Table 1: Evolution of VFH Performance Metrics (Selected Generations)

Generation	Impact Rate	Avg Targets	Avg Time
1	0.20	0.20	3.07
5	0.00	0.30	3.46
10	0.50	0.50	2.05
15	0.30	0.40	2.58
25	0.20	0.40	2.86
35	0.30	0.60	1.82

Parameter Evolution and Qualitative Behavior Analyzing the progression of the parameter vector $\boldsymbol{\theta}_{\text{vfh}}$ reveals significant shifts in the reactive control strategy. The initial parameter set was defined as:

$$[28.0, 3.0, 12.0, 8.0, 15.0, 3.0, 1.5, 5.0]$$

By Generation 35, the Evolution Strategies framework had converged on the following optimized values:

$$[34.33, 3.25, 10.04, 2.21, 3.00, 0.10, 0.79, 1.00]$$

These mathematical shifts appear to correlate with observable, qualitative changes in the AUV's navigational behavior:

- **Expanded Safety Bubble:** The safety distance threshold (d_{safe}) increased from an initial 28.0 to a final value of 34.33. This suggests the optimized parameters favor earlier obstacle detection and reaction, effectively expanding the protective spatial envelope.
- **Reduced Aggressiveness:** The thrust weight (W_{THRUST}) was aggressively minimized by the optimizer, dropping from an initial 5.0 down to the lower bound of 1.00. Similarly, the target direction weight ($W_{\text{TARGET_DIR}}$) decreased from 12.0 to 10.04. This suggests that the ES algorithm penalized aggressive, high-speed rushes toward the waypoint, favoring slower, more calculated maneuvers that reduced the impact rate (dropping to 0.0 impacts in generation 5).
- **Decreased Inertia:** The weight governing the preference to maintain the previous trajectory ($W_{\text{PREV_DIR}}$) plummeted from 3.0 to the lower bound of 0.10. This near-elimination of inertial preference allows the AUV to perform rapid, evasive steering adjustments without being artificially constrained by its past heading.
- **Diminished Repulsion:** Interestingly, the raw obstacle repulsion weight (W_{OBS}) also decreased significantly from 15.0 to 3.00. Combined with the increased safety distance, this suggests the agent relies on early detection and gradual steering rather than violent, late-stage repulsive corrections.

The optimization process achieved a better balance between the competing demands of obstacle avoidance and waypoint acquisition. While early generations struggled to reach targets (0.20 average targets in Generation 1), the optimized policy consistently reached higher target averages (up to 0.60 in Generation 35). Furthermore, the average mission time decreased, achieving 1.82 minutes by Generation 35, suggesting that the refined parameters resulted in more efficient, decisive paths despite the reduced thrust scaling.

5 Testing Phase: Multi-Agent Swarm Dynamics

5.1 Testing Procedure and Architectural Expansion

To evaluate the robustness of the optimized VFH parameters under high-stress, dynamic conditions, the experimental framework was expanded from a single-agent evaluation to a multi-agent swarm scenario. A dedicated Python orchestrator (`multi_tester_orchestrator.py`) was developed to manage the concurrent simulation of N autonomous Tethys AUVs operating simultaneously within the same Gazebo environment.

The testing orchestrator builds upon the core infrastructure used during training but introduces critical synchronizations to prevent computational bottlenecks. When scaling up to 15 concurrent AUVs, simulating 360 obstacles and computing 15 parallel 3D point cloud streams places an immense load on the CPU. If the physics engine were allowed to run unconstrained, the ROS 2 C++ nodes would lag behind the simulation time, leading to artificial collisions caused by computational desynchronization rather than algorithmic failure. To prevent this, the orchestrator executes the Gazebo server in headless mode while strictly locking the update rate to match the real-time factor (`--update-rate 1000`). Furthermore, the orchestrator implements a robust telemetry verification loop; it halts the deployment of the swarm and restarts the setup phase if any of the N drones fail to report their initial status via localized RAM files (`/tmp/status_tethys_X.txt`), guaranteeing a perfectly initialized simulation for every iteration.

5.2 Geometric Disparities: The Funnel Effect

The environment utilized for the multi-agent stress test (`generate_multi_world.py`) shares the same fundamental obstacle density as the training environment—deploying 100 spheres, 40 pillars, 120 boxes, and 100 beams[cite: 18, 19]. The safety protocol preventing obstacle generation within 20.0 meters of spawn points and waypoints is also maintained. However, the spatial distribution of the AUVs introduces a severe geometric challenge absent during the single-agent training phase.

During training, a single AUV was spawned at a random coordinate, and its target was placed at another fully random coordinate, allowing for diverse, unconstrained routing. In contrast, the multi-agent testing environment dictates a strictly convergent routing topology. The N drones are spawned evenly along the outer circumference of the arena, at a radius ranging between 90.0 and 130.0 meters. For every drone i, its target waypoint is mathematically constrained to be its exact diametric opposite across the map:

$$\text{Target}_{x,i} = -\text{Start}_{x,i} \tag{17}$$

$$\text{Target}_{y,i} = -\text{Start}_{y,i} \tag{18}$$

This specular target assignment guarantees that the intended linear paths of all N drones intersect exactly at the center of the arena (coordinates 0,0). Consequently, the swarm is forced into a spatial "funnel" or bottleneck. The drones must simultaneously navigate the static obstacles while actively avoiding the dynamic obstacles presented by the other converging AUVs.

To systematically analyze the limitations of the purely reactive VFH algorithm under these convergent conditions, four distinct stress tests were conducted. The tests evaluated swarms of 5, 10, 15, and 20 drones. Each configuration was executed for 50 iterations, with a maximum time allowance of 7 minutes per iteration, to collect statistically significant cumulative telemetry.

6 Results and Conclusions

The multi-agent testing phase systematically evaluated the ES-optimized VFH policy under scaling swarm densities. The cumulative results of the four testing configurations are presented in Table 2.

Table 2: Cumulative Swarm Stress Test Results

Swarm Size	Goal Reached	Obstacle Collisions	Drone Collisions	Timeouts
5 Drones	46.0%	34.4%	0.0%	19.6%
10 Drones	44.8%	31.4%	0.0%	23.4%
15 Drones	42.7%	28.0%	1.1%	28.3%
20 Drones	42.4%	29.6%	4.1%	23.9%

While the optimal policy demonstrated a 60% to 70% success rate during single-agent evolutionary training, its performance degraded consistently when scaled to the multi-agent funnel scenario. For a minimal swarm of 5 drones, the cumulative goal reach rate dropped to 46.0%. As the density increased to 20 concurrent AUVs, the overall success rate fell further to 42.4%.

Notably, drone-to-drone collisions remained statistically insignificant across all tests (peaking at a maximum of 4.1% in the 20-drone test). This indicates that the VFH algorithm was able to treat neighboring drones as dynamic obstacles and calculate repulsive vectors to avoid direct physical impacts. Instead, mission failures were dominated by two main factors: static obstacle collisions (averaging around 30% across all densities) and severe timeout rates.

The timeout rate directly correlates with swarm density, increasing from 19.6% in the 5-drone scenario to 28.3% in the 15-drone scenario. This reveals a primary limitation of the VFH in high-density environments: local minima stagnation. As multiple drones converge at the center of the arena, their mutual repulsion creates a localized, high-density obstacle barrier. Unable to formulate a global path around this dynamic cluster, the individual AUVs are repeatedly repulsed, entering cyclic holding patterns or U-shaped dead ends until the mission timer expires.

These testing results highlight an inherent constraint of purely reactive geometric algorithms. Even when perfectly parameterized through Evolution Strategies, the VFH algorithm is mathematically limited by its instantaneous perception. The primary deficiency observed is the phenomenon of *background interference*. Because the VFH evaluates a 2D spherical projection of the environment, a wall situated behind the target waypoint will lead to artificial repulsion away from valid targets. The algorithm lacks the contextual depth perception to distinguish that the waypoint is safely accessible before the obstacle.

References

[1] Johann Borenstein and Yoram Koren. The vector field histogram-fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation*, 7(3):278–288, 1991.

[2] X. Fan et al. The control method of autonomous flight avoidance barriers of 3d-vector field histogram (3d-vfh). *Journal of Autonomous Systems*, 2023.

[3] Thi Thoa Mac, Cosmin Copot, Duc Trung Tran, and Robin De Keyser. Heuristic approaches in robot path planning: A survey. *Robotics and Autonomous Systems*, 86:13–28, 2016.

[4] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.